

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN LẬP TRÌNH TÍNH TOÁN

CHƯƠNG TRÌNH QUẢN LÝ GIA PHẢ

Giảng viên hướng dẫn:

TS. Nguyễn Văn Hiệu

Sinh viên thực hiện:

Nguyễn Đình Hải Đăng

Lớp: 25T_KHDL

Lê Bá Sơn

Lớp: 25T_KHDL

Đà Nẵng, tháng 6 năm 2026

MỤC LỤC

Danh mục hình ảnh	iii
Mở đầu	1
1 TỔNG QUAN ĐỀ TÀI	2
2 CƠ SỞ LÝ THUYẾT	2
2.1 Ý tưởng	2
2.2 Cơ sở lý thuyết	3
2.2.1 Cấu trúc dữ liệu cây	3
2.2.2 Biểu diễn cây bằng mô hình con đầu - anh em kế tiếp	4
2.2.3 Duyệt cây và quản lý bộ nhớ	5
3 TỔ CHỨC CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN	6
3.1 Phát biểu bài toán	6
3.2 Cấu trúc dữ liệu	6
3.2.1 Thiết kế chung	6
3.2.2 Cài đặt các ràng buộc cần thiết	7
3.3 Thuật toán	8
3.3.1 Thuật toán Duyệt và Tìm kiếm	8
3.3.2 Thuật toán Xác định quan hệ	9
3.3.3 Thuật toán truy vấn theo danh xưng	11
3.3.4 Thuật toán lưu trữ và phục hồi	13
4 CHƯƠNG TRÌNH VÀ KẾT QUẢ	15
4.1 Tổ chức chương trình	15
4.1.1 Kiến trúc tổng thể của chương trình	15
4.1.2 Cấu trúc tệp tin và phân rã chức năng	15
4.2 Ngôn ngữ cài đặt	16
4.2.1 Lý do sử dụng ngôn ngữ C++	16
4.2.2 Môi trường phát triển và công cụ hỗ trợ	16
Tài liệu tham khảo	18

DANH MỤC HÌNH ẢNH

1	Hình minh hoạ mô hình biểu diễn cây theo cấu trúc con đầu - anh em kế tiếp	4
2	Mã giả mô tả thuật toán thêm một người con vào cây gia đình	5
3	Đoạn mã C++ thể hiện logic duyệt cây theo chiều sâu và giải phóng bộ nhớ.	6
4	Sơ đồ ERD mô tả thực thể Person và các mối quan hệ tự thân trong cấu trúc cây LCRS	7
5	Mô phỏng quy trình thực hiện truy vấn theo Định danh bằng cấu trúc Bảng băm	9
6	Mã giả và hình vẽ minh hoạ cho pha 1 của thuật toán.	10
7	Mã giả và hình vẽ minh hoạ cho pha 2 của thuật toán	10
8	Mã giả mô tả pha thứ nhất của thuật toán truy vấn theo danh xưng . . .	12
9	Mã giả mô tả pha thứ hai của thuật toán truy vấn theo danh xưng	13
10	Mã giả mô tả thuật toán phẳng hoá cây để lưu file	14
11	Sơ đồ mô tả cấu trúc chương trình và các tệp tin	16

MỞ ĐẦU

Trong bối cảnh công nghệ thông tin ngày càng phát triển mạnh mẽ, việc ứng dụng công nghệ vào giải quyết các bài toán thực tiễn đóng vai trò quan trọng. Trong đó, nhiều văn hoá truyền thống của dân tộc Việt Nam được “số hoá”, tạo cơ hội cho những người trẻ - những người sống trong thời đại công nghệ được tiếp cận với truyền thống văn hoá nước nhà theo một cách mới mẻ và thú vị hơn. Đây cũng chính là mục tiêu về lâu dài mà nhóm chúng em hướng tới khi thực hiện chuỗi các học phần PBL tại trường.

Xuất phát từ ý tưởng số hóa và tổ chức thông tin về các mối quan hệ huyết thống trong gia đình, nhóm chúng em lựa chọn thực hiện đề tài “**Chương trình Quản lý gia phả**”. Mục tiêu của chương trình là xây dựng một hệ thống cho phép lưu trữ, quản lý và truy vấn thông tin các thành viên trong gia đình dưới dạng cấu trúc cây, từ đó hỗ trợ thực hiện các thao tác như thêm thành viên, tìm kiếm quan hệ họ hàng, duyệt cây gia phả và hiển thị thông tin một cách có hệ thống. Thông qua việc xây dựng chương trình, nhóm có cơ hội vận dụng các kiến thức cơ bản về cấu trúc dữ liệu, thuật toán và lập trình để giải quyết bài toán quản lý thông tin có cấu trúc phân cấp - đúng như tinh thần của phương pháp *học tập qua dự án*.

Trong quá trình thực hiện dự án, nhóm đã nhận được sự hướng dẫn của TS. Nguyễn Văn Hiệu, giảng viên phụ trách học phần. Nhóm xin chân thành cảm ơn thầy đã định hướng và góp ý để nhóm có thể hoàn thành dự án này.

Mặc dù rất mong muốn thực hiện một chương trình hoàn chỉnh cả về nền tảng lý thuyết và phương pháp thực hiện, cũng như mong muốn có thêm phản hồi từ người dùng về chất lượng chương trình, tuy nhiên vì thời gian và trình độ còn hạn chế, vì vậy chương trình và báo cáo chắc chắn sẽ còn tồn tại những thiếu sót nhất định, kính mong quý thầy cô đóng góp ý kiến cho chương trình và báo cáo để nhóm rút kinh nghiệm cho những dự án PBL tiếp theo trong quá trình học tập tại trường.

Xin trân trọng cảm ơn.

1. TỔNG QUAN ĐỀ TÀI

Đề tài tập trung xây dựng một chương trình quản lý gia phả bằng ngôn ngữ C/C++, trong đó thông tin các thành viên và mối quan hệ huyết thống được mô hình hoá dưới dạng cấu trúc cây. Chương trình cho phép tạo lập, cập nhật, tra cứu và lưu trữ dữ liệu gia phả một cách có hệ thống.

Thông qua đề tài, sinh viên được rèn luyện kỹ năng phân tích bài toán thực tế, lựa chọn cấu trúc dữ liệu phù hợp và hiện thực hoá các chức năng cơ bản như thêm thành viên, duyệt cây và truy vấn quan hệ. Đề tài mang tính thực hành cao, phù hợp với mục tiêu của học phần PBL 1 và trình độ của sinh viên thực hiện.

2. CƠ SỞ LÝ THUYẾT

2.1. Ý tưởng

Về cấu trúc dữ liệu cốt lõi, nhóm sử dụng Cấu trúc liên kết động (Dynamic Linked Structure). Mỗi thành viên trong gia phả được đại diện bởi một nút (Node) trong bộ nhớ Heap.

Định nghĩa Nút (Person Node): Mỗi nút chứa các thành phần thông tin cá nhân (Data) và các thành phần liên kết (Pointers).

Cơ chế liên kết: Nhóm sử dụng mô hình Left-Child Right-Sibling (LCRS) để biểu diễn cây đa phân, cụ thể:

- `pLeftChild`: Con trỏ quản lý địa chỉ của người con đầu tiên.
- `pRightSibling`: Con trỏ quản lý địa chỉ của người em kế tiếp trong cùng một thế hệ.
- `pParent`: Con trỏ ngược về nút cha để tối ưu hóa thuật toán truy vết quan hệ (LCA).

Về quản lý bộ nhớ động, ý tưởng chính là tối ưu hóa tài nguyên hệ thống. Chỉ khi thêm một thành viên mới, chương trình mới sử dụng lệnh `new` để “xin” cấp phát bộ nhớ. Điều này giúp hệ thống có thể quản lý gia phả với quy mô không giới hạn (tùy thuộc vào RAM) thay vì bị giới hạn bởi kích thước mảng khai báo trước. Ngoài ra, nhóm cài đặt cơ chế đệ quy để thu hồi bộ nhớ (`delete`) khi xóa một nhánh hoặc khi kết thúc chương trình, đảm bảo không xảy ra hiện tượng rò rỉ bộ nhớ (Memory Leak).

Về các thuật sử dụng trên cây con trỏ:

1. Duyệt cây: Nhóm sử dụng thuật toán Duyệt theo chiều sâu (DFS) thông qua đệ quy để hiển thị sơ đồ gia phả. Việc di chuyển giữa các nút thực chất là việc truy cập vào địa chỉ bộ nhớ được lưu trữ trong các con trỏ liên kết.
2. Định vị đối tượng: Thay vì tìm kiếm theo chỉ số (Index), nhóm xây dựng hàm tìm kiếm trả về địa chỉ vùng nhớ của đối tượng. Địa chỉ này sau đó được dùng làm tham số đầu vào cho các thao tác cập nhật hoặc thêm con cháu, giúp tốc độ truy cập dữ liệu đạt mức tối ưu.

2.2. Cơ sở lý thuyết

2.2.1 Cấu trúc dữ liệu cây

Cây (Tree) là một cấu trúc dữ liệu phi tuyến tính, được sử dụng để biểu diễn các mối quan hệ phân cấp giữa các đối tượng. Trong cấu trúc cây, mỗi phần tử được gọi là một nút (node), các nút được liên kết với nhau thông qua quan hệ cha - con. Một cây có các đặc điểm cơ bản sau:

- Có một nút gốc (root), là nút không có cha.
- Mỗi nút (trừ nút gốc) có duy nhất một nút cha.
- Mỗi nút có thể có không hoặc nhiều nút con.
- Không tồn tại chu trình trong cấu trúc cây.

Nhờ đặc tính phân cấp rõ ràng, cấu trúc cây đặc biệt phù hợp để mô hình hoá các hệ thống có quan hệ thứ bậc như cây thư mục, sơ đồ tổ chức, và gia phả.

Trong bài toán quản lý gia phả, mỗi thành viên trong gia đình được biểu diễn bởi một nút trong cây, trong đó:

- Quan hệ cha-con tương ứng với quan hệ giữa các nút.
- Các thế hệ trong gia đình tương ứng với các mức (level) khác nhau của cây.
- Tổ tiên cao nhất được xem là nút gốc của cây gia phả.

Cách biểu diễn này cho phép mô hình hoá một cây nhiều nhánh với số lượng con bất kỳ, đồng thời đảm bảo cấu trúc dữ liệu gọn nhẹ, linh hoạt và thuận tiện cho việc cài đặt các thuật toán duyệt cây, thêm hoặc mở rộng gia phả trong ngôn ngữ C++.

2.2.2 Biểu diễn cây bằng mô hình con đầu - anh em kế tiếp

Đối với cây nhiều nhánh, mỗi nút có thể có số lượng nút con không xác định. Nếu lưu trực tiếp danh sách con cho mỗi nút, cấu trúc dữ liệu sẽ trở nên phức tạp và khó quản lý. Để khắc phục vấn đề này, đề tài sử dụng mô hình con đầu-anh em kế tiếp (first-child / next-sibling) để biểu diễn cây gia phả.

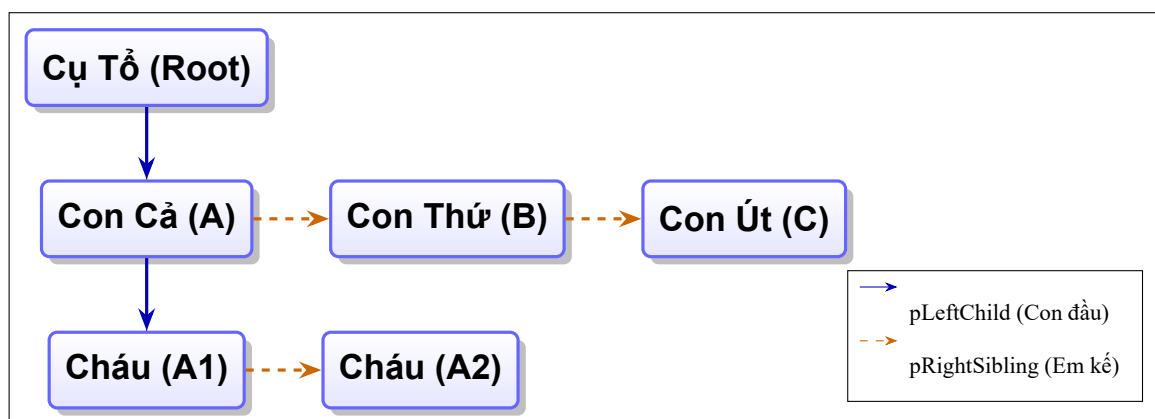
Trong mô hình này, mỗi nút trong cây chỉ cần hai liên kết chính:

- Con trỏ `firstChild`: trỏ tới nút con đầu tiên của nút hiện tại.
- Con trỏ `nextSibling`: trỏ tới nút anh/chị/em kế tiếp có cùng nút cha.

Ngoài ra, mỗi nút còn lưu con trỏ `parent` trỏ tới nút cha, giúp việc truy vấn quan hệ cha - con và di chuyển ngược trong cây trở nên thuận tiện.

Với cách biểu diễn này:

- Nếu một nút không có con, con trỏ `firstChild` có giá trị `nullptr`.
- Nếu một nút là người con cuối cùng, con trỏ `nextSibling` có giá trị `nullptr`.
- Các con của cùng một nút cha được liên kết thành một danh sách liên kết đơn thông qua `nextSibling`.



Hình 1: Hình minh họa mô hình biểu diễn cây theo cấu trúc con đầu - anh em kế tiếp

Trong chương trình, mỗi thành viên trong gia phả được biểu diễn bởi một cấu trúc `Person`, trong đó các mối quan hệ được quản lý hoàn toàn bằng con trỏ. Việc thêm một người con mới được thực hiện bằng cách:

- Nếu người cha chưa có con, gán trực tiếp nút mới cho `firstChild`.
- Nếu người cha đã có con, duyệt danh sách các con thông qua `nextSibling` để tìm người con cuối cùng (con út), sau đó nối nút mới vào cuối danh sách.

Mã giả 1 Thêm con vào danh sách anh chị em

```

1: child ← TAOĐOITUONG(Person, name, gender, parent)
2: if parent.firstChild = null then
3:     parent.firstChild ← child
4: else
5:     current ← parent.firstChild
6:     while current.nextSibling ≠ null do
7:         current ← current.nextSibling
8:     current.nextSibling ← child

```

▷ Nếu parent chưa có con đầu
▷ Gán child làm con đầu tiên của parent

▷ Nối child vào cuối danh sách con của parent

Hình 2: Mã giả mô tả thuật toán thêm một người con vào cây gia đình

Cách biểu diễn này mang lại các ưu điểm sau:

1. Biểu diễn được cây gia phả có số lượng con không giới hạn cho mỗi thành viên.
2. Cấu trúc dữ liệu gọn nhẹ, mỗi nút chỉ cần số lượng con trở cố định.
3. Thuận tiện cho việc duyệt cây theo chiều sâu và cài đặt các thuật toán xử lý quan hệ huyết thống.
4. Phù hợp với việc rèn luyện kỹ năng sử dụng con trỏ và quản lý bộ nhớ động trong C++.

Nhờ những đặc điểm trên, mô hình con đầu - anh em kế tiếp sử dụng con trỏ là lựa chọn phù hợp để hiện thực chương trình quản lý gia phả trong phạm vi đề tài.

2.2.3 Duyệt cây và quản lý bộ nhớ

Trong chương trình, việc xử lý và hiển thị gia phả được thực hiện thông qua duyệt cây theo chiều sâu, cụ thể là duyệt tiên thứ tự. Theo cách duyệt này, mỗi nút được xử lý trước, sau đó lần lượt duyệt đến nút con đầu tiên thông qua con trỏ `firstChild`, và tiếp tục duyệt các nút anh/chị/em kế tiếp thông qua con trỏ `nextSibling`.

Cách duyệt này rõ ràng phù hợp với mô hình con đầu - anh em kế tiếp và cho phép hiển thị gia phả theo thứ tự từ tổ tiên đến các thế hệ sau, phản ánh đúng cấu trúc phân cấp của gia đình.

Do chương trình sử dụng cấp phát bộ nhớ động cho các nút trong cây, việc quản lý và giải phóng bộ nhớ là yêu cầu bắt buộc. Khi kết thúc chương trình, toàn bộ các nút trong cây được giải phóng bằng một hàm đệ quy, đảm bảo không xảy ra rò rỉ bộ nhớ. Việc kết hợp duyệt cây và quản lý bộ nhớ động giúp chương trình hoạt động ổn định và an toàn trong môi trường C++.

Đến đây, những cơ sở lý thuyết đã được trình bày ở trên đã căn bản làm nền tảng cho


```

// Depth-First Traversal
void DisplayTree(Person* current, int level) {
    if (current == nullptr) return;
    // Child
    DisplayTree(current->firstChild, level + 1);
    // Sibling
    DisplayTree(current->nextSibling, level);
}

// Free up memory
void FreeTree(Person* current) {
    if (current == nullptr) return;
    FreeTree(current->firstChild);
    FreeTree(current->nextSibling);
    delete current;
}

```

Hình 3: Đoạn mã C++ thể hiện logic duyệt cây theo chiều sâu và giải phóng bộ nhớ.

toàn bộ quá trình thiết kế chương trình sau này.

3. TỔ CHỨC CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN

3.1. Phát biểu bài toán

Trong việc quản lý gia phả của một dòng họ, thách thức lớn nhất nằm ở tính phân cấp và sự biến động của dữ liệu. Cụ thể, bài toán đặt ra các yêu cầu và ràng buộc sau:

- Thứ nhất, một cá nhân có thể là con của người này nhưng lại là cha/mẹ của người khác. Mỗi quan hệ này kéo dài qua nhiều thế hệ, tạo thành một cấu trúc cây.
- Thứ hai, khác với cây nhị phân (mỗi nút chỉ có tối đa 2 con), một người trong thực tế có thể có số lượng con bất kỳ. Việc sử dụng mảng cố định để lưu trữ danh sách con sẽ gây lãng phí bộ nhớ hoặc thiếu linh hoạt.
- Thứ ba, cần lưu trữ rõ ràng các thông tin thực thể (Tên, giới tính, ngày sinh...) và các mối quan hệ (Cha-con, Anh-chị-em).
- Cuối cùng, bài toán yêu cầu các thao tác: thêm mới, duyệt và tìm kiếm.

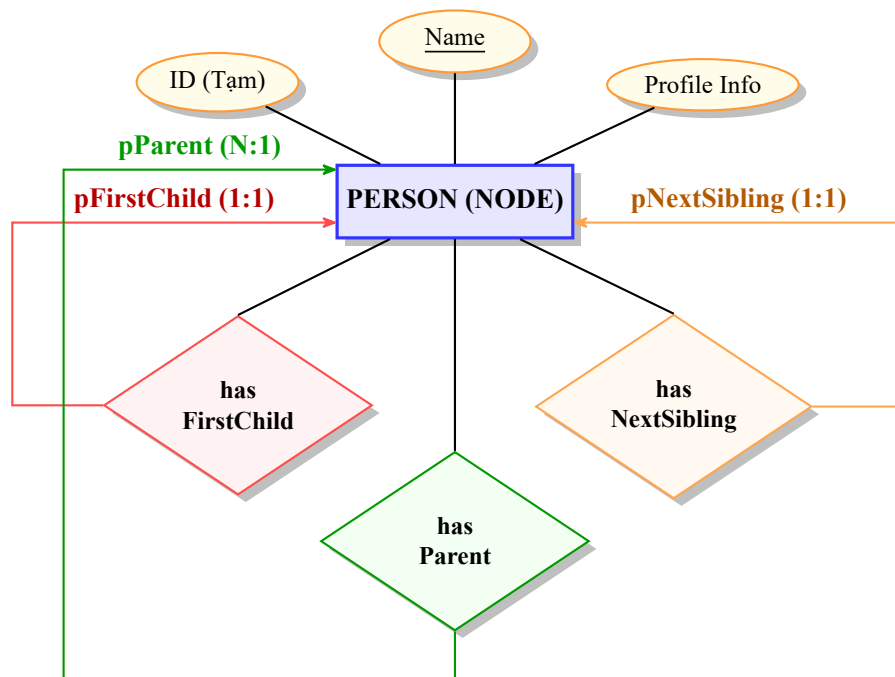
3.2. Cấu trúc dữ liệu

3.2.1 Thiết kế chung

Để giải quyết bài toán quản lý thông tin đa dạng của mỗi cá nhân trong dòng họ, chương trình thiết kế một kiểu dữ liệu tự định nghĩa (`struct person`). Sơ đồ cấu trúc được

mô tả ở Hình 4. Cấu trúc này ánh xạ trực tiếp các yêu cầu của thực tế vào không gian bộ nhớ:

- Nhóm thuộc tính định danh và nhân khẩu học: Các trường dữ liệu name, gender, birthday, job được sử dụng để lưu trữ hồ sơ cơ bản. Trong đó, gender (Nam/Nữ) đóng vai trò là cờ điều hướng (flag) để quyết định cấu trúc phát triển của cây.
- Nhóm thuộc tính gia đình và tình trạng: Trường spouseName (Tên vợ/chồng) được gắn trực tiếp như một thuộc tính của Nút thay vì tạo một Nút độc lập, giúp giảm độ phức tạp của đồ thị cây. Trường deathDay quản lý tình trạng sinh tử, và num-Children (số con) được thiết kế đặc biệt chỉ kích hoạt lưu trữ đối với thành viên Nữ.
- Nhóm thuộc tính định tuyến (Routing Attributes): Các con trỏ pParent, pFirstChild, pNextSibling đóng vai trò là “chỉ dẫn đường” để móc nối thực thể này vào mạng lưới gia phả chung, đảm bảo tính liên kết động mà không cần mảng trung gian.



Hình 4: Sơ đồ ERD mô tả thực thể Person và các mối quan hệ tự thân trong cấu trúc cây LCRS

3.2.2 Cài đặt các ràng buộc cần thiết

Cấu trúc dữ liệu đã được tùy biến để tuân thủ nghiêm ngặt các quy tắc truyền thống của một gia phả dòng họ, gồm hai ý chính:

1. Theo yêu cầu đề tài, con gái khi lập gia đình sẽ tính theo gia phả nhà chồng. Do

đó, trong giải thuật AddChild (Thêm con), nhóm đã cài đặt chốt chặn logic, nghĩa là: Khi người dùng thực hiện thao tác thêm thế hệ mới, chương trình sẽ kiểm tra parent->gender. Nếu giá trị là “Nữ”, thao tác bị từ chối và nút đó vĩnh viễn là một “Nút lá” (không bao giờ có pFirstChild).

2. Trong một dòng họ, việc trùng họ tên giữa các thế hệ là khá phổ biến (ví dụ: nhiều người cùng tên “Nguyễn Văn A”). Việc tìm kiếm và cập nhật của chương trình nếu chỉ dựa vào chuỗi name sẽ dẫn đến sai lệch nghiêm trọng. Để giải quyết, nhóm cơ chế Loại danh sách lựa chọn (Selection List). Thay vì định danh bằng chuỗi ký tự không duy nhất, chương trình sử dụng địa chỉ bộ nhớ làm định danh tuyệt đối, kết hợp với việc truy vấn thuộc tính phụ từ người dùng để xác nhận đối tượng.

3.3. Thuật toán

3.3.1 Thuật toán Duyệt và Tìm kiếm

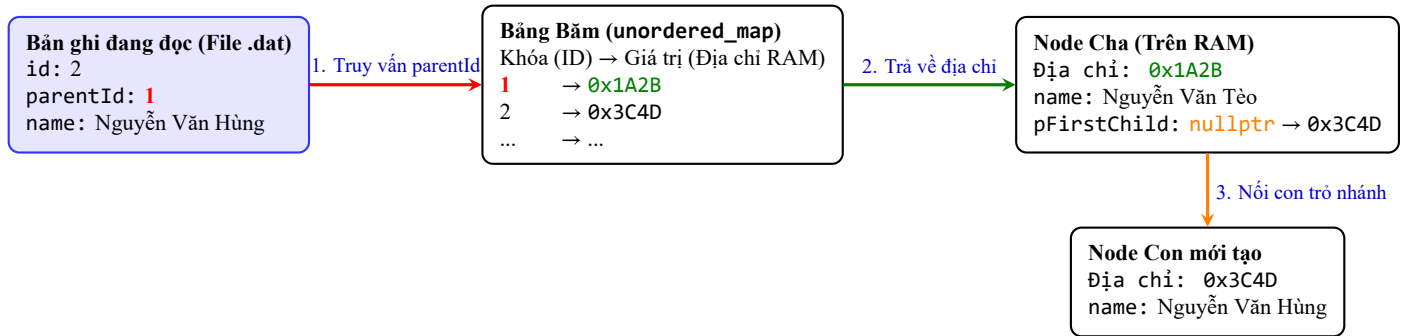
Vì các nút bộ nhớ (Person) được cấp phát động trên RAM, chương trình thiết kế hai chiến lược truy vấn chuyên biệt dựa trên loại khóa tìm kiếm để tối ưu hóa hiệu năng hệ thống:

Truy vấn theo Định danh (ID Lookup)

- *Cơ sở lý thuyết:* Sử dụng cấu trúc Bảng băm (Hash Table) nhằm thiết lập ánh xạ trực tiếp $f : ID \rightarrow \text{Memory Address}$, cho phép truy xuất dữ liệu độc lập với độ sâu của đồ thị cây.
- *Phương pháp thực hiện:* Kỹ thuật này được kích hoạt trong tiến trình đọc tệp nhị phân .dat. Chương trình khởi tạo `std::unordered_map<int, Person*>` để lưu trữ danh sách các nút vừa cấp phát. Khi cần khôi phục tập hợp cạnh E (nối con trở cha-con), hệ thống chỉ cần truyền `parentId` vào Bảng băm để nhận lại con trở thực thi ngay lập tức. Hình 5 mô tả dễ hiểu quá trình này.
- *Phân tích độ phức tạp:* Thuật toán đưa độ phức tạp thời gian tìm kiếm cha từ $O(|V|)$ xuống $O(1)$ tuyệt đối, giải quyết triệt để nút thắt cổ chai khi hệ thống phải tái thiết lập hàng vạn liên kết con trở.

Truy vấn theo Tên gọi

- *Cơ sở lý thuyết:* Do thuộc tính “Tên” là dữ liệu chuỗi phi tuyến tính (không tạo thành cây tìm kiếm nhị phân BST), hệ thống triển khai thuật toán Duyệt theo chiều sâu (Depth-First Search - DFS).



Hình 5: Mô phỏng quy trình thực hiện truy vấn theo Định danh bằng cấu trúc Bảng băm

- *Phương pháp thực hiện:* Thuật toán sử dụng đệ quy Tiền thứ tự (Pre-order), duyệt tuần tự qua các liên kết pFirstChild và pNextSibling. Để khắc phục điểm yếu của thuật toán tham lam (Greedy) truyền thống, chương trình không trả về kết quả khớp đầu tiên. Thay vào đó, thuật toán tiếp tục duyệt cạn không gian đỉnh để thu thập toàn bộ các thực thể trùng tên vào một `std::vector<Person*>`. Khối kết quả này sau đó được đẩy lên tầng giao diện (UI) để người dùng phân biệt và chọn lựa mục tiêu chính xác.
- *Phân tích độ phức tạp:* Độ phức tạp thời gian là $O(|V|)$ (số lượng thành viên). Độ phức tạp không gian lưu trữ cho ngăn xếp gọi hàm (Call Stack) là $O(H)$, với H là chiều cao cực đại của cây gia phả. Với quy mô dữ liệu thực tế của một dòng họ, thuật toán đảm bảo tốc độ phản hồi ở mức thời gian thực.

3.3.2 Thuật toán Xác định quan hệ

Bài toán Xác định quan hệ và danh xưng trong gia phả thực chất là một bài toán kinh điển của cấu trúc dữ liệu cây. Dưới góc độ cơ sở toán học trong khoa học máy tính, đây chính là sự kết hợp giữa thuật toán Tìm tổ tiên chung gần nhất (Lowest Common Ancestor - viết tắt là LCA) và việc phân tích Ma trận độ lệch khoảng cách.

Để đảm bảo thuật toán chạy chính xác, cũng như bao quát được sự phức tạp của hệ thống danh xưng trong tiếng Việt, thuật toán sẽ trải qua bốn pha (phase) như sau:

Pha thứ nhất: Truy vết đường đi về gốc (Path tracing).

- Mục tiêu của pha: Tìm chuỗi các đỉnh kết nối từ người đang xét lên tới Cụ Tổ, làm cơ sở cho việc xác định LCA ở pha thứ hai.
- Phương pháp thực hiện: Giả sử đồ thị cây gia phả là $T = (V, E)$, ta cần xây dựng hai tập hợp đường đi cho nút A và nút B . Quy trình thực hiện như sau:
 - Từ con trở của A , dùng vòng lặp di chuyển theo hướng con trở pParent cho

đến khi gặp `nullptr` (Nút gốc).

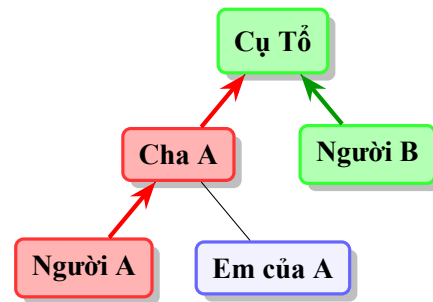
- Lưu lại các địa chỉ bộ nhớ đã đi qua vào một mảng (hoặc `std::vector`). Ta thu được tập $P_A = \{A, P_{A1}, P_{A2}, \dots, \text{Root}\}$.
- Làm tương tự với B , ta thu được tập $P_B = \{B, P_{B1}, P_{B2}, \dots, \text{Root}\}$.
- Về độ phức tạp: Độ phức tạp thời gian xấp xỉ $O(H)$, với H là chiều cao lớn nhất của cây (số lượng thế hệ). Thao tác này cực kỳ nhanh vì nó chỉ đi theo một đường thẳng lên trên, không cần duyệt qua các nhánh khác của gia phả; Độ phức tạp không gian là $O(H)$ vì cần cấp phát hai mảng để lưu trữ tối đa H con trỏ địa chỉ bộ nhớ cho mỗi đường đi.

Mã giả 2 GetPathToRoot(Node)

```

Khởi tạo mảng Path rỗng
while Node không phải là rỗng do
    Thêm Node vào đầu mảng Path (để Gốc luôn ở vị trí số 0)
    Node = Node.parent
return Path

```



Hình 6: Mã giả và hình vẽ minh họa cho pha 1 của thuật toán.

Pha thứ hai: Xác định Tổ tiên chung gần nhất (LCA).

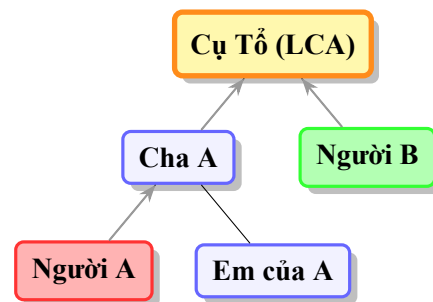
- Mục tiêu của pha: Tìm ra nút chung đầu tiên mà cả hai đối tượng A và B cùng thuộc về nhánh hậu duệ của nút đó.
- Phương pháp thực hiện: Ta thực hiện phép so sánh hai tập P_A và P_B đã có được từ pha thứ nhất, cụ thể:
 - So sánh hai danh sách P_A và P_B từ vị trí nút gốc (đầu danh sách).
 - Nút cuối cùng trùng khớp trong cả hai danh sách trước khi đường đi rẽ nhánh chính là LCA cần tìm.
- Về độ phức tạp: Độ phức tạp thời gian là $O(K)$, với $K = \min(h_A, h_B)$, phép so sánh được thực hiện trực tiếp trên địa chỉ bộ nhớ thay vì so sánh chuỗi.

Mã giả 3 DetermineRelationship (A, B)

```

Đặt PathA = GetPathToRoot(A)
Đặt PathB = GetPathToRoot(B)
for i from 0 to min(h_A, h_B) do
    if PathA_i trùng PathB_i then
        Đặt LCAIndex = i
    else break
if LCAIndex = -1 then
    return Không cùng huyết thống
else return LCAIndex

```



Hình 7: Mã giả và hình vẽ minh họa cho pha 2 của thuật toán

Pha thứ ba: Tính toán véc-tơ khoảng cách.

- Mục tiêu của pha: Định lượng khoảng cách thế hệ giữa các đối tượng và tổ tiên chung để làm cơ sở cho việc phân giải danh xưng.
- Phương pháp thực hiện: Gọi d_A là khoảng cách thế hệ từ LCA đến A. Gọi d_B là khoảng cách thế hệ từ LCA đến B. Lúc này, ta có một cặp tọa độ (d_A, d_B) phản ánh cấp bậc của hai người, làm tiền đề cho pha cuối cùng.
- Về độ phức tạp: Vì mảng đường đi chứa tuần tự các đỉnh từ Gốc đến nút đích, khoảng cách d được chứng minh bằng công thức toán học tuyệt đối: $d = (M - 1) - T$ với M là chiều dài mảng và T là vị trí của LCA. Vậy độ phức tạp thời gian và không gian cho pha này chỉ là $O(1)$.

Pha cuối cùng: Phân giải danh xưng

- Mục tiêu của pha: Chuyển đổi các giá trị toán học (d_A, d_B) thành tên gọi quan hệ họ hàng theo quy chuẩn văn hóa.
- Phương pháp thực hiện: Sử dụng cấu trúc điều kiện để ánh xạ cặp (d_A, d_B) vào ma trận danh xưng, cụ thể:

Tọa độ (d_A, d_B)	Quan hệ của B đối với A	Quan hệ của A đối với B	Giải thích Logic
($k, 0$)	Tổ tiên đời thứ k	Hậu duệ đời thứ k	$B=L$, B là gốc của nhánh chứa A.
($0, k$)	Hậu duệ đời thứ k	Tổ tiên đời thứ k	$A=L$, A là gốc của nhánh chứa B.
(k, k)	Anh/Chị/Em họ đời $k-1$	Anh/Chị/Em họ đời $k-1$	Chung tổ tiên cách đây k thế hệ.
($1, 2$)	Bác/Cô/Chú/Dì	Cháu (gọi bằng Bác/Chú...)	B là con của anh/chị/em với A.
($2, 1$)	Cháu (gọi bằng Bác/Chú...)	Bác/Cô/Chú/Dì	A là con của anh/chị/em với B.
($1, k$) với $k > 2$	Cụ/Kỵ (nhánh bàng hệ)	Chắt/Chút (nhánh bàng hệ)	B là hậu duệ xa của anh/chị/em với A.
(d_A, d_B)	Họ hàng tổng quát	Họ hàng tổng quát	Cách nhau tổng $d_A + d_B$ bậc thế hệ.

3.3.3 Thuật toán truy vấn theo danh xưng

Một bài toán khác được đặt ra ngược với bài toán được xác định ở Mục 3.3.2, đó là tìm người theo quan hệ, nghĩa là: cho một người xác định và quan hệ (bố mẹ/ông bà/cô chú/...), ta cần phải tìm được một (hoặc nhiều) người có quan hệ đó với người đã xác định. Hệ thống tận dụng tính đối ngẫu của thuật toán LCA trước đó. Thay vì tìm tọa độ

từ hai nút, ta sẽ cố định tọa độ (d_A, d_B) dựa trên danh xưng tiếng Việt và thực hiện truy vết ngược từ con trở pParent kết hợp duyệt nhánh ngang pNextSibling để trả về kết quả cần thiết. Cụ thể, thuật toán này trải qua ba pha như sau:

Pha thứ nhất: Xác định tổ tiên.

- Mục tiêu của pha: Tìm ra nút tổ tiên chung (L).
- Phương pháp thực hiện: Từ nút A , di chuyển qua con trở pParent đúng d_A lần. Ví dụ: Tìm “Chú” thì $d_A = 2$. Từ A , ta cần leo lên “Bố” ($d = 1$), rồi sau đó lên “Ông nội” ($d = 2$). Nút Ông nội chính là L .
- Về độ phức tạp: Độ phức tạp thời gian là $O(d_A)$, vì chỉ đi theo một đường thẳng nên rất nhanh.

Mã giả 4 FindAncestor(startP, dA)

```

1: temp ← startP
2: for  $i = 1 \rightarrow dA$  do
3:   if temp.parent ≠ NULL then
4:     temp ← temp.parent
5:   else return NULL
6: return temp

```

▷ Không đủ độ sâu thế hệ

Hình 8: Mã giả mô tả pha thứ nhất của thuật toán truy vấn theo danh xưng

Pha thứ hai: Duyệt và lọc đối tượng.

- Mục tiêu của pha: Thu thập được những người phù hợp.
- Phương pháp thực hiện: Từ tổ tiên L , ta cần tìm các hậu duệ ở mức thế hệ d_B thỏa mãn các điều kiện về giới tính và thứ tự. Ta sẽ sử dụng phương pháp duyệt theo chiều rộng (BFS) để tìm tất cả các nút cách L đúng d_B bậc.
- Về độ phức tạp: Độ phức tạp thời gian là $O(S)$, với S là số lượng anh/em/con/cháu trong nhánh đang xét đó.

Mã giả mô tả thuật toán của pha ở hình 9.

Pha thứ ba: Phân giải danh xưng cụ thể.

- Mục tiêu của pha: Áp dụng những ràng buộc đã có để đưa ra người cần tìm kiếm.
- Phương pháp thực hiện: Đây là lúc áp dụng các ràng buộc mà nhóm đã thiết kế trong struct Person, cụ thể là phân giải theo “Giới tính” và “Thứ tự”. Cụ thể:
 - Theo Giới tính: Nếu yêu cầu là “Cô”, lọc những người có gender là “Nữ”.
 - Theo Thứ tự: Dựa vào con trở pNextSibling. Nếu người đó nằm phía trước Cha của A trong danh sách anh em, đó là “Bác”, ngược lại là “Chú”.

Mã giả 5 Thuật toán thu thập ứng viên cùng thế hệ

```

1: function COLLECTCANDIDATES(root, dB)
2:   Cands  $\leftarrow \emptyset$  ▷ Khởi tạo tập ứng viên rỗng
3:   if dB = 0 then
4:     return {root} ▷ Nếu khoảng cách = 0, trả về chính gốc
5:   procedure DUYETTANG(curr, depth)
6:     if curr = NULL then
7:       return ▷ Trường hợp cơ sở: không có node để xử lý
8:     if depth = dB then
9:       Cands  $\leftarrow$  Cands  $\cup$  {curr} ▷ Thêm node hiện tại vào tập ứng viên
10:      return ▷ Dừng duyệt sâu hơn
11:      DUYETTANG(curr.firstChild, depth + 1) ▷ Duyệt node con đầu tiên (xuống sâu hơn)
12:      t  $\leftarrow$  curr.firstChild ▷ Bắt đầu duyệt các node anh em
13:      while t  $\neq$  NULL and t.nextSibling  $\neq$  NULL do
14:        t  $\leftarrow$  t.nextSibling ▷ Chuyển sang node anh em kế tiếp
15:        DUYETTANG(t, depth + 1) ▷ Duyệt cây con của node anh em này
16:      DUYETTANG(root, 0) ▷ Bắt đầu duyệt từ gốc ở độ sâu 0
17:      return Cands ▷ Trả về tất cả các ứng viên đã thu thập được

```

Hình 9: Mã giả mô tả pha thứ hai của thuật toán truy vấn theo danh xưng

Tổng quát, thuật toán chung của ta vẫn tiệm cận mức $O(H)$, rõ ràng là hiệu quả hơn nhiều so với việc quét toàn bộ người trong gia phả. Ngoài ra, thuật toán này còn có ưu điểm là xử lý được trường hợp đa kết quả (ví dụ: tìm “Chú” có thể cho ra nhiều người, việc tìm người cụ thể do người dùng quyết định).

3.3.4 Thuật toán lưu trữ và phục hồi

Trong môi trường quản lý dữ liệu bằng con trỏ động, địa chỉ bộ nhớ của các thực thể (Person) chỉ có giá trị nội hàm trong phiên làm việc hiện tại của RAM. Để đảm bảo tính bền vững của dữ liệu, nhóm áp dụng kỹ thuật Tuần tự hóa nhị phân (Binary Serialization) kết hợp với thuật toán Phẳng hóa đồ thị cây.

Cấu trúc Bản ghi tĩnh trung gian (PersonRecord)

Để khắc phục nhược điểm của chuỗi động `std::string`, chương trình định nghĩa một cấu trúc `PersonRecord` kích thước cố định, sử dụng mảng ký tự tĩnh (`char[]`). Các liên kết con trỏ LCRS (`pParent`, `pFirstChild`, `pNextSibling`) được thay thế hoàn toàn bằng định danh số nguyên là `id` (mã cá nhân) và `parentId` (mã của nút cha).

Giải thuật Ghi tệp (Phẳng hóa cây LCRS)

- Hệ thống sử dụng thuật toán duyệt cây Tiên thứ tự (Pre-order Traversal) thông qua hàm đệ quy `FlattenTree`.

- Tại mỗi nút, dữ liệu được sao chép an toàn (sử dụng `strncpy` để chống tràn bộ đệm) sang cấu trúc `PersonRecord` và đẩy vào một mảng động `std::vector`.
- Thay vì ghi tuần tự từng cá nhân, chương trình áp dụng kỹ thuật Block I/O: tính toán tổng dung lượng của mảng và sử dụng phương thức `write()` để đổ toàn bộ khối dữ liệu từ RAM xuống tệp `.dat` trong một chu kỳ ghi duy nhất.

Mã giả 6 Hàm FlattenTree

Require: `currentNode`, `recordArray`

```

1:                                     ▷ Điều kiện dừng đệ quy
2: if currentNode là RÕNG then return
3:                                     ▷ 1. Chuyển đổi Node động thành Bản ghi tĩnh
4: Tạo NewRecord
5: NewRecord.id = currentNode.id
6: if currentNode có Cha then
7:   NewRecord.parentId = currentNode.parent.id
8: else
9:   NewRecord.parentId = 0                                     ▷ 0 ám chỉ đây là Cội Tổ (Gốc)
10: Chép các thông tin (Tên, Năm sinh...) từ Node sang Bản ghi
11:                                     ▷ 2. Lưu vào mảng
12: Thêm NewRecord vào recordArray
13:                                     ▷ 3. Đệ quy đi tiếp xuống nhánh Con và nhánh Em
14: FlattenTree(currentNode.firstChild, recordArray)
15: FlattenTree(currentNode.nextSibling, recordArray)

```

Hình 10: Mã giả mô tả thuật toán phẳng hoá cây để lưu file***Giải thuật Đọc tệp và Tái cấu trúc***

Quá trình phục hồi dữ liệu từ tệp nhị phân được tối ưu hóa ở mức cao nhất nhằm tránh việc tìm kiếm lặp lại:

1. Chương trình đọc toàn bộ khối dữ liệu từ tệp lên một mảng `PersonRecord` trên RAM.
2. Khởi tạo một Bảng băm (Hash Map) sử dụng cấu trúc `std::unordered_map<int, Person*>` để thiết lập quan hệ ánh xạ giữa `id` nguyên thủy và địa chỉ bộ nhớ động vừa được cấp phát.
3. Khi duyệt qua từng bản ghi, hệ thống tạo mới một đối tượng `Person` và sử dụng `parentId` để tra cứu trực tiếp địa chỉ người cha trong Bảng băm. Thao tác tra cứu này đạt độ phức tạp thời gian $O(1)$, cho phép liên kết con trở ngay lập tức thông qua hàm `AddChild` mà không cần duyệt lại cây LCRS.
4. Hệ thống tự động cập nhật lại biến đếm ID toàn cục (`global_id_counter`) dựa trên mức ID lớn nhất trong tệp để đảm bảo tính duy nhất cho các phiên nhập liệu tiếp theo.

Đánh giá thuật toán

Cả hai tiến trình Serialization và Deserialization đều đạt độ phức tạp thời gian $O(N)$ và không gian $O(N)$, với N là tổng số thành viên trong gia phả. Việc kết hợp Block I/O và Hash Map giúp giải thuật xử lý mượt mà dữ liệu quy mô hàng vạn thực thể mà không gây ra tình trạng tràn ngăn xếp (Stack Overflow) hay nghẽn cổ chai truy xuất đĩa.

4. CHƯƠNG TRÌNH VÀ KẾT QUẢ**4.1. Tổ chức chương trình****4.1.1 Kiến trúc tổng thể của chương trình**

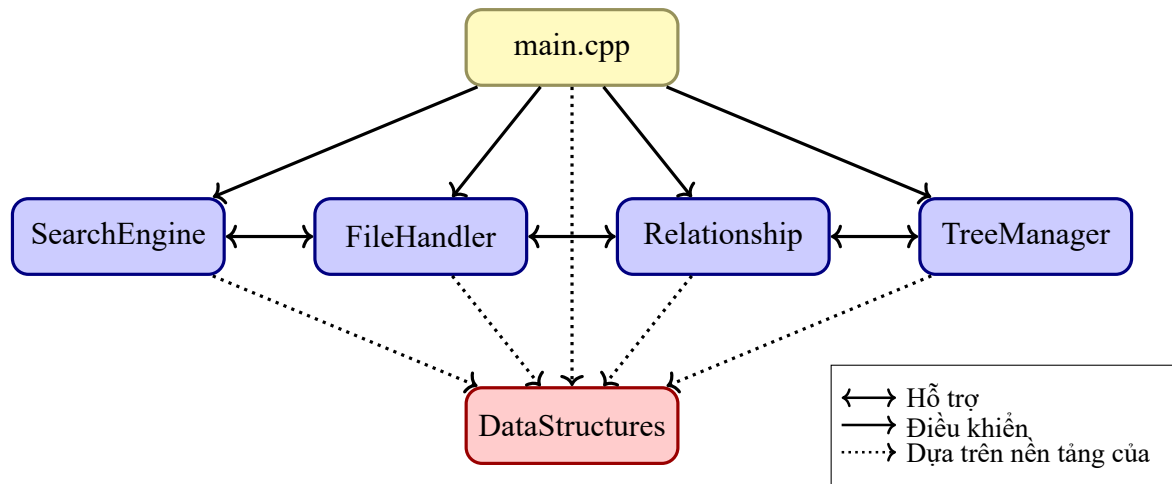
Hệ thống được thiết kế và xây dựng dựa trên mô hình Lập trình theo mô-đun (module). Thay vì tập trung toàn bộ mã nguồn vào một tệp duy nhất, chương trình được phân rã thành các mô-đun độc lập dựa trên *nguyên lý Phân tách trách nhiệm*.

Cách tiếp cận này đảm bảo hệ thống đạt được độ gắn kết cao bên trong từng mô-đun và độ phụ thuộc thấp giữa các mô-đun với nhau, tạo tiền đề thuận lợi cho việc kiểm thử, gỡ lỗi và mở rộng tính năng trong tương lai.

4.1.2 Cấu trúc tệp tin và phân rã chức năng

Mã nguồn của đồ án được tổ chức thành 6 thành phần cốt lõi, bao gồm 1 tệp tiêu đề cơ sở, 4 cặp module logic (.h và .cpp) và một tệp điều khiển trung tâm. Chi tiết chức năng của từng tệp được quy định như sau:

- (a) `DataStructures.h`: Định nghĩa cấu trúc nút LCRS, bản ghi và biến toàn cục, đảm bảo nhất quán kiểu dữ liệu giữa các mô-đun.
- (b) `TreeManager.h/.cpp`: Thao tác trực tiếp trên RAM; khởi tạo biến toàn cục; quản lý vòng đời nút (cấp phát, giải phóng, truy xuất).
- (c) `SearchEngine.h/.cpp`: Xử lý truy vấn, kiểm soát trùng lặp; duyệt DFS để tìm kiếm; cung cấp giao diện chọn thực thể.
- (d) `Relationship.h/.cpp`: Giải quyết bài toán đồ thị gia phả; tìm tổ tiên chung (LCA); đổi tọa độ thế hệ sang danh xưng và ngược lại.
- (e) `FileHandler.h/.cpp`: Quản lý I/O với ổ cứng; tuần tự hóa cây LCRS thành mảng, ghi/đọc file nhị phân; hỗ trợ nhập từ file văn bản, xử lý lỗi dòng.
- (f) `main.cpp`: Khởi chạy, tạo dữ liệu mẫu, điều hướng sự kiện và xử lý lỗi nhập.



Hình 11: Sơ đồ mô tả cấu trúc chương trình và các tệp tin

Toàn bộ mã nguồn của chương trình quản lý gia phả có tại Phụ lục đính kèm theo báo cáo này.

4.2. Ngôn ngữ cài đặt

4.2.1 Lý do sử dụng ngôn ngữ C++

Dự án lựa chọn ngôn ngữ C++ làm ngôn ngữ lập trình chính để triển khai hệ thống bởi những lý do như sau:

- Mô hình LCRS yêu cầu cấp phát bộ nhớ liên tục, C++ dùng new/delete để kiểm soát vòng đời của từng nút (Person) một cách chính xác, nhờ đó tối ưu hiệu suất.
- Dùng `std::string` thay vì mảng char giúp xử lý tên tiếng Việt an toàn, tránh tràn bộ đệm. `std::vector` cũng được dùng để lưu kết quả tìm kiếm, giúp mã nguồn ngắn gọn, rõ ràng.
- Nhờ struct/class, C++ mô hình hóa “Con người” tự nhiên với đầy đủ thuộc tính và quan hệ, đồng thời dễ dàng tách chương trình thành các tệp .h và .cpp.

4.2.2 Môi trường phát triển và công cụ hỗ trợ

- Trình biên dịch: Sử dụng GCC (g++) tiêu chuẩn C++11 trở lên để hỗ trợ các tính năng hiện đại và đảm bảo tốc độ thực thi tối ưu.
- Hệ thống được phát triển trên các IDE phổ biến như *Embarcadero Dev-C++* và *Visual Studio*, tận dụng khả năng quản lý *project* để liên kết các tệp tin mô-đun.
- Định dạng tệp lưu trữ: tệp nhị phân (.dat) dùng để lưu trữ trạng thái cây dưới dạng các struct tĩnh, đảm bảo tốc độ đọc/ghi nhanh và bảo mật dữ liệu cơ bản; tệp văn

bản (.txt) Dùng để nhập liệu hàng loạt theo định dạng phân tách bằng dấu phẩy (CSV), giúp người dùng dễ dàng soạn thảo dữ liệu từ bên ngoài.

TÀI LIỆU THAM KHẢO

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
- [2] Đặng Thiên Bình (n.d.). *Bài giảng Cấu trúc dữ liệu*. Lưu hành nội bộ.
- [3] chahelgupta. (n.d.). *DSA-Project-FamilyTree: C++ program for building and visualizing a hierarchical family tree*. GitHub. <https://github.com/chahelgupta/DSA-Project-FamilyTree>
- [4] praveenmylavarapu. (n.d.). *family-tree: A family tree program in C++ using n-ary tree*. GitHub. <https://github.com/praveenmylavarapu/family-tree>
- [5] Stack Overflow contributors. (2012). *Data structure for family tree*. Stack Overflow. <https://stackoverflow.com/questions/8487374/data-structure-for-family-tree>
- [6] tenouk. (n.d.). *Constructing a very simple family structure tree using the C++ inheritance principles*. tenouk.com. <https://www.tenouk.com/cpluspluscodesnippet/cplusplusclassinheritancefamilytree.html>
- [7] Trường Đại học Quy Nhơn. (2025). *Mẫu code C++ cho cây nhị phân và cây gia phả*. Studocu. <https://www.studocu.vn/vn/document/dai-hoc-quy-nhon/li-luan-giang-day/mau-code-c-cho-cay-nhi-phan-va-cay-gia-pha-kcj/148579038>
- [8] homielab. (n.d.). (2026). *Gia Phả OS (Gia Phả Open Source)*. Github. <https://github.com/homielab/giapha-os>
- [9] Wikipedia contributors. (2025). *Kinship*. Wikipedia, The Free Encyclopedia. <https://en.wikipedia.org/wiki/Kinship>